

Avaliação de Microarquitetura RISC-V com o simulador gem5

Descrição. No trabalho anterior, utilizamos um simulador (Venus, Ripes, etc) para validar a corretude funcional de um algoritmo em assembly RISC-V. Neste trabalho, o objetivo é analisar o desempenho desse algoritmo de acordo com diferentes aspectos microarquiteturais utilizando o **gem5**. Os objetivos específicos são, com apresentação de um seminário e relatório científico, compreender técnicas modernas de processamento fora de ordem, hierarquia de memória e medidas de desempenho.

Serão ofertados diferentes trabalhos, onde cada grupo deve escolher um tópico. A única restrição é que não haja o mesmo algoritmo sendo avaliado no mesmo tópico. Assuntos distintos podem ser discutidos diretamente com o professor. Cada grupo deverá pesquisar e explicar o conceito relacionado, assim como fazer uma avaliação de performance usando seu código do trabalho 1 usando os conceitos pesquisados. Os seguintes assuntos serão discutidos:

- **Comparação de modelos:** Comparação direta de pipeline (In-Order) com out-of-order. Esse trabalho trará uma explicação teórica e prática sobre o funcionamento do modelo out-of-order.
- **Branch Prediction:** Comparar técnicas de branch prediction no modo pipeline; o grupo deve alternar o preditor entre dois extremos: LocalBP (preditor local simples) e BiModeBP (preditor global complexo).
- **Superescalaridade:** Avaliação de superescalaridade do processador ao aumentar a largura dos estágios. Alterar simultaneamente a largura dos estágios (fetchWidth, decodeWidth, issueWidth, commitWidth) de 1 (escalar) para 4 (superescalar), observando o impacto direto no desempenho.
- **Avaliação de desvios no out-of-order (OoO):** Avaliar o custo de desvios mal previstos alterando a qualidade do preditor (comparando LocalBP vs BiModeBP no modelo OoO) e medindo especificamente a métrica de instruções descartadas (*squashed instructions*) comparada com as instruções efetivadas (*committed*). Qual se adequa melhor ao seu algoritmo?
- **Políticas de Substituição de Cache:** Comparação de políticas de substituição de cache (ex: LRU vs Random) e avaliar qual delas se adapta melhor ao padrão de acesso à memória do algoritmo.
- **Associatividade de cache:** Alterar os parâmetros de mapeamento da memória cache (ex: comparar cache Direta com cache Totalmente Associativa) e identificar qual se adequa melhor ao seu algoritmo.
- **Janela de Instruções (ROB - Reorder Buffer):** Variar o tamanho do ROB (ex: de 64 para 128 e 256 entradas).

Tópico	Cenários Conceituais a Comparar	O que deve ser medido (Métricas)
1. Comparação de Modelos	Processador tradicional em-ordem (In-Order) vs. Processador fora-de-ordem (Out-of-Order).	• Tempo total de simulação • Instruções por ciclo (IPC)
2. Branch Prediction	Preditor de desvios simples vs. Preditor de desvios complexos (ambos no modelo em-ordem).	• Total de desvios avaliados • Quantidade de erros de predição
3. Superescalaridade	Processador escalar (largura igual a 1) vs. Processador superescalar (largura igual a 4).	• Instruções por ciclo (IPC) • Tempo total de simulação
4. Desvios no Out-of-Order	Preditor de desvios simples vs. Preditor complexo (ambos rodando no modelo fora-de-ordem)	• Instruções confirmadas com sucesso • Instruções descartadas (especulação errada)
5. Políticas de Substituição	Política baseada em uso recente (temporal) vs. Política de escolha aleatória.	• Quantidade total de falhas (misses) na cache de dados • Taxa de erro da cache
6. Associatividade de Cache	Cache com mapeamento direto vs. Cache com alta associatividade por conjuntos.	• Quantidade total de falhas (misses) na cache de dados • Tempo total de simulação
7. Janela de Instruções (ROB)	Processador com janela de instruções estreita vs. Processador com janela de instruções ampla.	• Instruções por ciclo (IPC) • Tempo total de simulação

Orientação: Os simuladores usados no primeiro trabalho possuem um ambiente de execução próprio com chamadas de sistema simplificadas. Para realizar experimentos no gem5, será necessário adaptar o código assembly para a sintaxe do GCC (ex: ajustar diretivas, garantir o ponto de entrada main ou _start). O gem5 também vai esperar um binário estático, portanto utilizaremos a compilação do gcc para gerar um binário estático.

Será disponibilizado um *script* de configuração em Python para o gem5 com duas configurações base de CPU:

- MinorCPU: Um modelo de CPU *In-Order* detalhado (representando o Pipeline tradicional).
- DerivO3CPU: Um modelo de CPU *Out-of-Order* (OoO) detalhado e superescalar.

Vocês deverão modificar os parâmetros do *script* fornecido para realizar os testes específicos para seu tema. Após cada simulação, o gem5 gera um documento chamado stats.txt.. Vocês deverão configurar variações da microarquitetura estudada e realizar

diferentes cargas de trabalho. Cada A avaliação do relatório dependerá da capacidade de localizar, extrair e explicar as métricas importantes para seu assunto. Exemplos incluem:

- `sim_ticks`: O tempo total de simulação (em picosegundos).
- `system.cpu.ipc`: O número de instruções executadas por ciclo.
- `system.cpu.branchPred.lookups` e `.condIncorrect`: Total de desvios e quantos foram mal previstos.
- `system.cpu.dcache.overall_misses`: O número total de falhas na cache de dados.

Entrega:

1. O código assembly adaptado e o comando utilizado para a compilação.
2. Relatório em PDF contendo gráficos comparativos das métricas extraídas do `stats.txt` para cada cenário.
3. Haverá uma entrega parcial, valendo 3 pontos, no dia 09 de Junho, de seu código sendo processado no `gem5`. A entrega final será no dia 23 de Junho, incluindo o relatório de até 4 páginas no formato da SBC. Após isso iniciarão as apresentações.